LAPORAN PRATIKUM

ALGORITMA DAN STRUKTUR DATA

Analisis Kompleksitas Algoritma

OLEH:

NAMA : Adinda Riesya Putri

NIM : 2025573010039

KELAS : TI 1D

DOSEN : M.Reza Zulman S.S.T., M.Sc

PROGRAM STUDI TEKNIK *INFOR*MATIKA

JURUSAN TEKNOLOGI *INFOR*MASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2026/2027

LEMBARAN PENGESAHAN

No Praktikum	:	04/TIK/TI 1D/STRUKTUR DATA DAN ALGORITMA/2026
Nama Praktikum	:	Praktikum Struktur Data dan Algoritma
Pembuat Laporan	:	Adinda Riesya Putri
NIM	:	2025573010039
Kelas	:	TI-1D
Jurusan	:	Teknologi Informasi dan Komputer
Prodi	:	Teknik Informatika
Tanggal Praktikum	:	23 April 2026
Tanggal Penyerahan Laporan	:	29 April 2026

Mengetahui

Dosen Pengajar,

M.Reza Zulman S.S.T., M.Sc

NIP: 199205012022031005

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum yang berjudul **" Analisis Kompleksitas Algoritma"** dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu tugas dalam mata kuliah **Algoritma dan Struktur Data** pada Program Studi Teknik Informatika. Dalam laporan ini penulis membahas tentang konsep Kompleksitas Algoritma, Big O Notation, Time Complexity, Space Complexity, Kelas-Kelas Kompleksitas Big O, dan Refacrtoring Algoritma.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun dari pembaca agar laporan ini dapat menjadi lebih baik di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep Object Oriented Programming dan struktur data dalam JavaScript.

Lhokseumawe 28 April 2026

Adinda Riesya Putri

Nim. 2025573010039

Daftar Isi

KATA PENGANTAR ..... **ii**

Daftar Isi ..... **iii**

BAB I PENDAHULUAN **1**

1.1 LATAR BELAKANG 1

1.2 TUJUAN PRAKTIKUM 1

1.3 RUMUSAN MASALAH 2

BAB II LANDASAN TEORI **3**

2.1 Analisis Kompleksitas Algoritma 3

2.2 Big O Notation 3

2.3 Time Complexity 4

2.4 Space Complexity 4

***2.5* Kelas-Kelas Kompleksitas Big O** 5

2.6 Refactoring Algoritma 5

BAB III LANGKAH-LANGKAH PRAKTIKUM **7**

3.1 Alat dan Bahan 7

3.2 Pembuatan *File* dan *Folder* Menggunakan *GitBash* 7

3.2 Percobaan dan Latihan 8

3.2.1 *Intro* kompleksitas 8

3.2.2 Kelas Big O 8

3.2.3 Latihan 1. 9

3.2.4 Space Complexity 9

3.2.5 *Refactor* 11

3.3 Tugas 12

3.3.1 Analisis dan Refactor 12

3.3.2 Visualisasi Pertembuhan Big O 13

3.4 Mengirim Praktikum Ke *GitHub* 13

BAB IV PEMBAHASAN DAN ANALISA **15**

4.1. Alat dan Bahan 15

4.2. Pembuatan File dan Folder 15

4.3. Percobaan dan Latihan 15

4.3.1Intro Kompleksitas 15

4.3.2Kelas *Big O* 15

4.3.3Latihan 1 16

4.3.4*Space Komplexity* 16

4.3.5*Refactcoring* 16

4.4. Tugas 16

4.4.1Analisis dan Refactoring 16

4.4.2Visualisasi Pertumbuhan Big O 16

4.5. Mengirim Praktikum ke Github 16

BAB V PENUTUP **17**

5.1 Kesimpulan 17

5.2 Saran 17

DAFTAR PUSTAKA **18**

BAB I
PENDAHULUAN

1.1 LATAR BELAKANG

Dalam pengembangan program, efisiensi algoritma merupakan aspek yang sangat krusial untuk diperhatikan. Sebuah algoritma yang berfungsi dengan baik pada data kecil belum tentu tetap efisien saat diterapkan pada data yang sangat besar. Oleh karena itu, diperlukan metode untuk mengukur seberapa baik kinerja suatu algoritma, baik dari segi waktu eksekusi maupun penggunaan memori.

Konsep analisis kompleksitas algoritma adalah proses untuk menilai seberapa efisien sebuah algoritma dalam menyelesaikan masalah, terutama dari perspektif waktu eksekusi (time complexity) dan penggunaan memori (space complexity). Analisis ini tidak tergantung pada kecepatan komputer, melainkan pada cara kerja algoritma seiring dengan bertambahnya ukuran input. Hal ini membantu programmer dalam membandingkan berbagai algoritma, memilih solusi yang paling efisien, dan menghindari program yang lambat serta boros sumber daya.

Salah satu metode yang paling umum digunakan dalam analisis kompleksitas adalah notasi Big O. Big O digunakan untuk menggambarkan tingkat pertumbuhan waktu atau memori yang diperlukan oleh algoritma berdasarkan ukuran input (n). Big O berfokus pada performa terburuk (worst-case) agar algoritma tetap aman digunakan dalam berbagai kondisi. Dengan Big O, programmer dapat memprediksi kinerja algoritma tanpa perlu menghitung waktu dalam satuan detik.

Selain itu, penelitian mengenai kompleksitas waktu juga menunjukkan bahwa perbedaan algoritma sangat memengaruhi kinerja program. Sebagai contoh, algoritma dengan kompleksitas O(n^2) akan memerlukan waktu lebih lama dibandingkan algoritma dengan kompleksitas O(n) ketika ukuran input semakin besar. Hal ini membuktikan bahwa pemilihan algoritma yang tepat sangat penting dalam pengembangan perangkat lunak yang efisien.

1.2 TUJUAN PRAKTIKUM

Ada pula tujuan praktikum kali ini sebagai berikut:

1. Memahami konsep Analisis Kompleksitas Algoritma.
2. Memahami penggunaan *Big O Notation* dalam mengukur efisiensi algoritma.
3. Menganalisis *time complexity* dan *space complexity* dalam mengukur efisiensi algoritma.
4. Mengidentifikasi tingkat kompleksitas dari berbagai algoritma,
5. Melakukan *refactoring code* agar program menjadi lebih optimal dan efisien.

1.3 RUMUSAN MASALAH

Ada pula rumusan masalah dalam praktikum ini:

1. Apa yang dimaksud dengan analisis kompleksitas algoritma?
2. Apa yang dimaksud dengan Big O Notation?
3. Bagaimana cara menentukan *Time complexity* dan *Space Complexity?*
4. Bagaimana melakukan *refactoring code* agar algoritma menjadi lebih efisien?

BAB II
LANDASAN TEORI

2.1 Analisis Kompleksitas Algoritma

Analisis kompleksitas adalah cara untuk mengukur efisiensi suatu algoritma berdasarkan waktu eksekusi dan penggunaan memori. Tujuan dari analisis ini adalah untuk mengetahui seberapa baik performa suatu program ketika ukuran input semakin besar. Algoritma yang terlihat cepat pada data kecil belum tentu tetap efisien pada data yang besar. Oleh karna itu, analisis kompleksitas sangat penteing dalam pengembangan perangkat lunak.

Analisis kompleksitas membantu programmer dalam menentukan algoritma terbaik dengan melihat jumlah operasi yang dilakukan program, bukan hanya hasil akhirnya. Analisis ini digunakan agar program tetap berjalan optimal meskipun jumlah data meningkat sangat besar.

Kompleksitas algoritma umumnya dibagi menjadi dua bagian utama, yaitu time complexity dan space complexity. Keduanya digunakan untuk menilai kualitas suatu algoritma secara lebih objektif.

2.2 Big O Notation

Big O Notation adalah notasi yang digunakan untuk menggambarkan laju pertumbuhan waktu atau memori yang dibutuhkan suatu algoritma terhadap ukuran input. Big O tidak mengukur waktu absolut dalam detik, tetapi mengukur growth rate dari algoritma tersebut. Hal ini penting karena waktu absolut sangat bergantung pada perangkat keras yang digunakan.

Big O digunakan untuk membandingkan efisiensi beberapa algoritma dalam menyelesaikan masalah yang sama. Dengan menggunakan Big O, seorang programmer dapat mengetahui algoritma mana yang lebih baik digunakan untuk data kecil maupun data besar.

Aturan penyederhanaan Big O:

1. Menghilangkan konstanta, contoh nya seperti O(2^n) menjadi O(n).
2. Menghilangkan suku kecil, contoh nya O(n^2 + n) menjadi O(n^2).
3. Memperhatikan suku yang paling dominan, semakin besar nilai n, suku terbesar menjadi penentu utama kompleksitas.

Contoh umum Big O:

- O(1) – konstan.
- O(log n) – logaritmik.
- O(n) – linear.
- O(n log n) – linearitmik.
- O(n^2) – kuadratik.
- O(2^n) – eksponensial.

2.3 Time Complexity

Tme Complexity adalah ukuran yang menunjukkan seberapa cepat waktu eksekusi suatu algoritma bertambah ketika ukuran input meningkat. Time complexity digunakan untuk membandingkan efisiensi antar algoritma dan membantu programmer memilih solusi yang paling optimal. Time complexity tidak menghitung waktu dalam satuan deik, tetapi menghitung jumlah operasi yang dilakukan oleh algoritma.

Time complexity menggambarkan hubungan antara ukuran input (n) dengan jumlah langkah yang dibutuhkan algoritma untuk menyelesaikan suatu proses. Semakin kecil nilai kompleksitas waktu, maka semakin efisien algoritma tersebut untuk data besar.

Contoh nya:

- Mengakses elemen array – O(1)
- Linear search – O(n).
- Binary search – O(log n).
- Bubble sort – O(n^2).
- Fibonacci rekursi – O(n^2).

Algoritma linear search harus memeriksa data satu per satu sehingga kompleksitasnya O(n), sedangkan binary search lebih cepat karena membagi data menjadi dua bagian setiap langkah sehingga kompleksitasnya O(log n). oleh karna itu, pemilihan algoritma sangat berpengaruh terhadap performa program.

2.4 Space Complexity

Space complexity merupakan ukuran memori tambahan yang dibutuhkan oleh algoritma selama proses eksekusi. Space complexity digunakan untuk mengetahui seberapa besar penggunaan memori suatu program selain data utama yang digunakan sebagai input.

Space complexity terdiri dari total memory usage dan auxiliary space. Namun, yang biasanya dianalisis adalah auxiliary space, yaitu memori tambahan yang digunakan selama program berjalan, seperti variabel sementara, array tambahan, struktur data baru, dan penggunaan call stack pada rekursi.

Contoh :

- Hanya menggunakan beberapa variabel → O(1)
- Membuat array baru sebesar n → O(n)
- Rekursi sedalam n tingkat → O(n) karena menggunakan call stack

Algoritma yang hanya menggunakan variabel sederhana memiliki space complexity O(1), sedangkan algoritma yang membuat array baru sesuai ukuran input akan memiliki kompleksitas O(n). Dalam beberapa kasus, terjadi trade-off antara waktu dan memori, yaitu algoritma yang lebih cepat membutuhkan memori yang lebih besar.

2.5 Kelas-Kelas Kompleksitas Big O

Big O Notation memiliki beberapa kelas kompleksitas yang digunakan untuk mengukur tingkat efisiensi suatu algoritma. Semakin kecil nilai kompleksitasnya, maka semakin baik performa algoritma tersebut ketika jumlah data semakin besar. Pemahaman terhadap kelas-kelas Big O Notation sangat penting agar programmer dapat memilih algoritma yang paling optimal sesuai kebutuh program.

Berikut urutan kompleksitas Big O dari yang paling cepat hingga paling lambat:

1. O(1) – konstan.
   Waktu eksekusi tidak bergantung pada ukuran input. Berapa pun jumlah data, waktu yang dibutuhkan tetap sama. Contoh penggunaan nya adalah pada mengakses elemen array berdasarkan index.
2. O(log n) – logaritmik.
   Masalah diperkecil menjadi setengah pada setiap langkah sehingga proses menjadi sangat efisien untuk data besar. Contoh penggunaan nya ada pada binary search.
3. O(n) – linear.
   Waktu eksekusi bertambah sebanding dengan jumlah data yang diproses. Contoh penggunaan nya ada pada linear search.
4. O(n log n) – linearitmik.
   Kompleksitas ini biasanya digunakan pada algoritma sorting yang efisien. Contoh nya ada digunakan pada merge sort dan quick sort.
5. O(n^2) – kuadratik.
   Terjadi ketika terdapat nested loop atau perulangan bersarang dua tingkat. Semakin besar data, waktu yang dieksekusi meningkat dengan cepat. Contoh penggunaan nya terdapat pada Bubble Sort.
6. O(2^n) – eksponensial.
   Merupakan kompleksitas yang sangat lambat karena jumlah proses bertambah dua kali lipat setiap penambahan input. Biasa nya hanya cocok untuk data kecil. Contoh penggunaan nya ada pada Fibonacci Rekursif.

2.6 *Refactoring Algoritma*

Refactoring adalah proses yang bertujuan untuk memperbaiki struktur kode agar lebih efisien tanpa mengubah hasil akhir dari program. Dalam analisis kompleksitas, refactoring sering dilakukan untuk mengubah algoritma yang lambat menjadi lebih optimal. Tujuan dari refactoring adalah untuk meningkatkan keterbacaan kode, mempermudah pemeliharaan program, serta meningkatkan performa aplikasi secara keseluruhan.

Code refactoring adalah teknik yang sangat penting dalam pengembangan perangkat lunak karena membantu programmer dalam mengurangi kode yang berulang, memperbaiki struktur program, dan meningkatkan efisiensi sistem tanpa mengubah fungsi utama dari

program. Dengan refactoring yang baik, aplikasi dapat beroperasi lebih cepat, lebih stabil, dan lebih mudah untuk dikembangkan di masa depan.

Contoh yang dapat diberikan adalah mengganti nested loop dengan penggunaan struktur data seperti Set atau Map sehingga kompleksitas dapat berubah dari $O(n^2)$ menjadi $O(n)$. Penggunaan struktur data yang tepat dapat secara signifikan meningkatkan performa program.

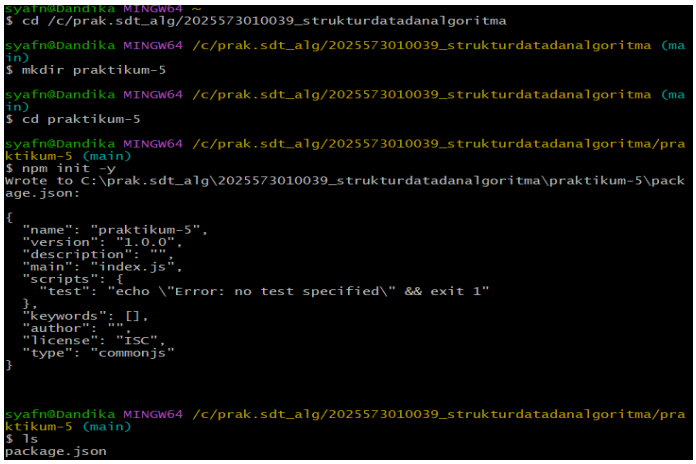
BAB III

LANGKAH LANGKAH PRAKTIKUM

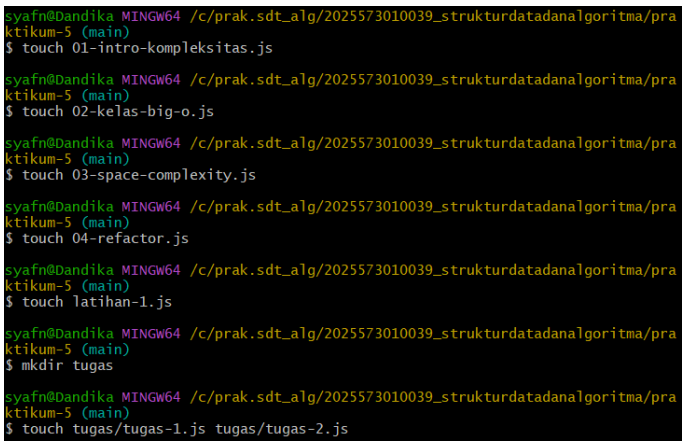
3.1 Alat dan Bahan

1. *GitBash.*
2. *Visual Studio Code.*
3. Modul praktikum.

3.2 Pembuatan *File* dan *Folder* Menggunakan *GitBash*



Gambar 1.0 Membuat *Folder* untuk Praktikum-5



Gambar 1.1 Membuat *File* dengan *GitBash* Buat Praktikum-5

3.2 Percobaan dan Latihan

3.2.1 *Intro* kompleksitas

```javascript
function ukurWaktu(label, fn){
    const mulai = Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label}: ${selesai - mulai} ms`);
}
const n = 100_000;

function jumlahkanLinear(n){
    let total = 0;
    for (let i = 1; i <= n; i++) total +=i;
        return total;
}

function jumlahkanRumus(n){
    return (n * (n + 1)) / 2;
}

function cariPasangan(arr) {
    const pasangan = [];
    for (let i = 0; i < arr.length; i++){
        for (let j = i + 1; j < arr.length; j++){
            if(arr[i] + arr[j] === 10) pasangan.push([arr[i], arr [j]]);
        }
    }
    return pasangan;
}

const data = Array.from({length : 5000}, (_, i) => 1);

console.log('perbandingan waktu ekssekusi');
ukurWaktu('O(1)  jumlahkanRumus ', () => jumlahkanRumus(n));
ukurWaktu('O(n)  jumlahkanLinear ', () => jumlahkanLinear(n));
ukurWaktu('O(n^2)  cariPasangan  ', () => cariPasangan(data));

console.log('hasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
```

Gambar 3.2.1.0 Kodingan Intro Kompleksitas

```
perbandingan waktu ekssekusi
O(1)  jumlahkanRumus : 0 ms
O(n)  jumlahkanLinear : 2 ms
O(n^2)  cariPasangan  : 15 ms
hasil sama? true
```

Gambar 3.2.1.1 Hasil Koding Intro Kompleksitas.

3.2.2 Kelas Big O

```javascript
function ambilPertama(arr) {return arr [0];}
function tambahkanItem(arr, item ) {arr.push(item);}
function isGenap(n) {return n % 2 === 0;}

function binarySearch(arr, target){
    let kiri = 0, kanan = arr.length - 1;
    let langkah = 0;
    while (kiri <= kanan){
        const tengah = Math.floor((kiri + kanan) / 2 );
        if (arr[tengah] === target){
            console.log(`ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
            return tengah;
        }
        if (arr[tengah] < target ) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}

function cariMax(arr){
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}

function bubbleSortNaif(arr){
    const a = [...arr];
    for (let i = 0; i < a.length - 1 -1; i++)
        for(let j = 0; j < a.length - i - 1; j++)
            if (a[j] > a[j+1] [a[j]], a[j+1] = [a[j+1], a[j]]);
    return a;
}

function fibRekursif(n){
    if (n <= 1) return n;
    return fibRekursif(n-1) + fibRekursif(n-2);
}
```

```javascript
console.log('O(1) - selalu cepat');
console.log(ambilPertama([10,20,30,40,50]));
console.log(isGenap(42));

console.log('O(log n) - binary search');
const sorted = Array.from({length:1_000_000}, (_,i)->i);
binarySearch (sorted, 731_452);

console.log('O(n) - linear search');
console.log('max dari 1000 elemen: ',
    cariMax(Array.from({length:1000}, ()->Math.random()*1000|0)));

console.log('O(n^2) - Bubble sort (kecil saja)');
console.log(bubbleSortNaif([64,34,25,12,22,11,90]));

console.log('O(2^n) - fibonacci rekursi (n kecil!');
for(let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
console.log(' ');

console.log('waktu fib(35) O(n) O(2^n): ');
let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');

console.log('waktu fib(35) O(n) memoization:');
const memo = {};
function fibMemo(n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
}
t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');
```

Gambar 3.2.2.0 Kodingan kelas Big O

```
O(1) - selalu cepat
10
true
O(log n) - binary search
ditemukan di indeks 731452 setelah 0 langkah
O(n) - linear search
max dari 1000 elemen:  999
O(n^2) - Bubble sort (kecil saja)
[
  64,
  [ [ [Array], 64 ], 64 ],
  [ [ [Array], [Array] ], [ [Array], 64 ] ],
  [ [ [Array], [Array] ], [ [Array], [Array] ] ],
  [ [ [Array], [Array] ], [ [Array], [Array] ] ],
  [ [ 11, [Array] ], [ [Array], [Array] ] ],
  [ 90, [ 11, [Array] ] ]
]
O(2^n) - fibonacci rekursi (n kecil!
0 1 1 2 3 5 8 13 21 34 55
waktu fib(35) O(n) O(2^n):
102 ms
waktu fib(35) O(n) memoization:
0 ms
```

Gambar 3.2.2.1 Hasil Kod*ing*

3.2.3 Latihan 1.

```javascript
function jumlahArray(arr){
 //fungsi A
 function fnA(n){
     return n * 2;
} }
 // big o, karena operasi nya hanya 1 kali saja, dan tidak berganti kepada n

 //fungsi b
 function fnB(n){
     for (let i = 0; i < n; i++){
         for(let j = 0; j < n; j++){
             console.log(i,j);
             }
         }
} }

 //big o(n^2), karena perulangan bersarang, n x n

 // fungsi c
 function fnC(n){
     for(let i = 1; i < n; i *= 2){
         console.log(i);
         }
 }

 //big o(log n), karena i dikali kan 2 setiap iterasi nya

 //fungsi d
 function fnD(arr){
     arr.forEach(x => {
         arr.forEach(y =>{
             arr.forEach(z => {
                 console.log(x,y,z);
                 });
             });
     });
 }

 //big O(n^3), 3 loop bersarang, n x n x n
```

```javascript
//mengimplementasikan
function hitungKompleksitas(n, fn){
    const start = Date.now();

    fn(n);

    const end = Date.now();
    console.log(`waktu eksekusi: ${end - start} ms`);
}

function fnB(n){
    for (let i = 0; i < n; i++){
        for (let j = 0; j < n; j++){

        }
    }
}

function fnc(n){
    for(let i = 1; i < n; i *= 2){}
}

function fnD(arr){
    arr.forEach(x => {
        arr.forEach(y =>{
            arr.forEach(z => {

            });
        });
    });
}

console.log('fungsi a');
hitungKompleksitas(1000,fnA);

console.log('fungsi B');
hitungKompleksitas(1000,fnB)

console.log('fungsi c');
hitungKompleksitas(1000,fnC)

console.log('fungsi d');
hitungKompleksitas(1000, (n) => {
    const arr = new Array(n).fill(0);
    fnD(arr);
});
```

Gambar 3.2.3.0 Koding Latihan 1

```
fungsi a
waktu eksekusi: 0 ms
fungsi B
waktu eksekusi: 1 ms
fungsi c
1
2
4
8
16
32
64
128
256
512
waktu eksekusi: 4 ms
fungsi d
waktu eksekusi: 969 ms
```

Gambar 3.2.3.1 Hasil Kod*ing*

3.2.4 Space Complexity

```javascript
function jumlahArray(arr){
    let total = 0;
    for (const x of arr) total += x;
    return total;
}

function duplikasiArray(arr){
    const baru = [];
    for(const x of arr) baru.push(x * 2);
    return baru;
}

function faktorialRekursif(n){
    if(n <= 1) return 1;
    return n *faktorialRekursif(n - 1);
}

function faktorialIteratif(n){
    let hasil = 1;
    for (let i = 2; i < n; i++) hasil *= i;
    return hasil;
}

const arr = [1,2,3,4,5,6,7,8,9,10];

console.log('jumlah array : ', jumlahArray(arr));
console.log('duplikasi array :', duplikasiArray(arr));
console.log('faktorial 10 rekursif :', faktorialRekursif(10));
console.log('faktorial 10 iteratif :', faktorialIteratif(10));

function hitungUnik(arr){
    const seen = new Set();
    for (const x of arr ) seen.add(x);
    return seen.size;
}

const dataAcak = [1,2,3,2,1,4,5,6,4,7];
console.log('elemen unik :', hitungUnik(dataAcak));
```

Gambar 3.2.4.1 Kod*ing Inheritance*

```
jumlah array :  55
duplikasi array : [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
faktorial 10 rekursif : 3628800
faktorial 10 iteratif : 362880
elemen unik : 7
```

Gambar 3.2.4.2 Hasil Kod*ing*

```
console.log('----------------------------------------latihan 3----------------------------------------');
// big O(n^2), karena dua loop berserang
//big O space, O(1) karena tidak menggunakan struktur data tambahan seperti Set
function cariPasanganLambat(arr, target){
    for (let i = 0; i < arr.length; i++){
        for(let j = i + 1; j < arr.length; j++){
            if (arr[i] + arr[j] === target){
            return [arr[i], arr[j]];
            }
        }
    }
    return null;
}

// big O(n), karena hanya ada satu loop
//big O space, O(n) karena menggunakan Set untuk menyimpan data nya.

function cariPasanganCepat(arr, taget){
    const set = new Set();
    for(let num of arr){
        const pasangan = target - num;
        if(set.has(pasangan)){
            return [pasangan, num];
        }
        set.add(num);
    }
    return null;
}


const data = [2,7,11,15];
const target = 9;

console.log('lambat :', cariPasanganLambat(data, target));
console.log('cepat :', cariPasanganCepat(data,target));

function ukurWaktu(fn, arr, target){
    const start = Date.now();
    fn(arr, target);
    const end = Date.now();
    return end - start;
}
```

```
const arrBesar = Array.from({length: 5000}, () => Math.floor(Math.random() * 100000));
const targetBesar = -1;

console.log('lambat: ', ukurWaktu(cariPasanganLambat, arrBesar, targetBesar), 'ms');
console.log('cepat: ', ukurWaktu(cariPasanganCepat, arrBesar, targetBesar), 'ms');

console.log('bisa dilihat dari hasil nya bahwa cariPasanganCepat lebih baik.');
console.log('secara teori pun, big O(n) lebih cepat daripada Big O(n^2)');
console.log('---trade off nya atau kelebihan dan kekurangan nya---');
console.log('cariPasanganLambat memiliki keuntungan, dia tidak membutuhkan memori tambahan. namun dia sangat lambat untuk data yang besar');
console.log('cariPasnaganLambat cocok untuk cata yang kecil. karena simpel dan data yang ada dikit jadi hanya membutuhkan sedikit waktu untuk eksekusi nya');
console.log('sedangkan cariPasanganCepat memiliki keuntungan dimana dia itu cepat untuk data yang besar. namun membutuhkan memori tambahan yaitu Set. ');
console.log('carPasanganCepat cocok untuk data yang besar karna dia tidak mencari nilai yang ada satu per satu melain kan  ');
```

Gambar 3.2.4.3 Latihan Koding Space Complexity

```
lambat : [ 2, 7 ]
cepat : [ 2, 7 ]
lambat:  41 ms
cepat:  1 ms
bisa dilihat dari hasil nya bahwa cariPasanganCepat lebih baik.
secara teori pun, big O(n) lebih cepat daripada Big O(n^2)
---trade off nya atau kelebihan dan kekurangan nya---
cariPasanganLambat memiliki keuntungan, dia tidak membutuhkan memori tambahan. namun dia sangat lambat untuk data yang besar
cariPasnaganLambat cocok untuk cata yang kecil. karena simpel dan data yang ada dikit jadi hanya membutuhkan sedikit waktu untuk eksekusi nya
sedangkan cariPasanganCepat memiliki keuntungan dimana dia itu cepat untuk data yang besar. namun membutuhkan memori tambahan yaitu Set.
carPasanganCepat cocok untuk data yang besar karna dia tidak mencari nilai yang ada satu per satu melain kan
```

Gambar 3.2.4.4 Hasil Kod*ing*

3.2.5 *Refactor*

```
function adaDuplikatLambat(arr){
    for (let i = 0; i < arr.lenght; i++ )
        for (let j = i + 1; j < arr.lenght; j++)
            if(arr[i] === arr[j]) return true;
    return false;
}

function adaDuplikatCepat(arr){
    const seen = new Set();
    for (const x of arr){
        if (seen.has(x)) return true;
        seen.add(x);
    }
    return false;
}

function frekuensiLambat(arr){
    const unik = [];
    const hitung = [];
    for (const x of arr){
        const idx = unik.indexOf(x);
        if (idx === -1) { unik.push(x); hitung.push(1);}
        else hitung [idx]++;
    }
    return Object.fromEntries(unik.map((u,i) => [u, hitung [i]]))
}

function frekuensiCepat(arr){
    const counter = {};
    for (const x of arr) counter[x] = (counter [x] || 0) + 1;
    return counter;
}

const besar =  Array.from({length: 20000}, () => Math.floor(Math.random() * 1000));

let t = Date.now();
adaDuplikatLambat(besar);
console.log('duplikat lambat O(n^2) : ', Date.now()-t, 'ms');

t = Date.now();
adaDuplikatCepat(besar);
console.log('duplikat cepat O(n) :', Date.now()-t, 'ms');

const kata = ['a', 'b', 'a', 'c', 'b', 'a', 'd'];
console.log('frekuensi :', frekuensiCepat(kata));
```

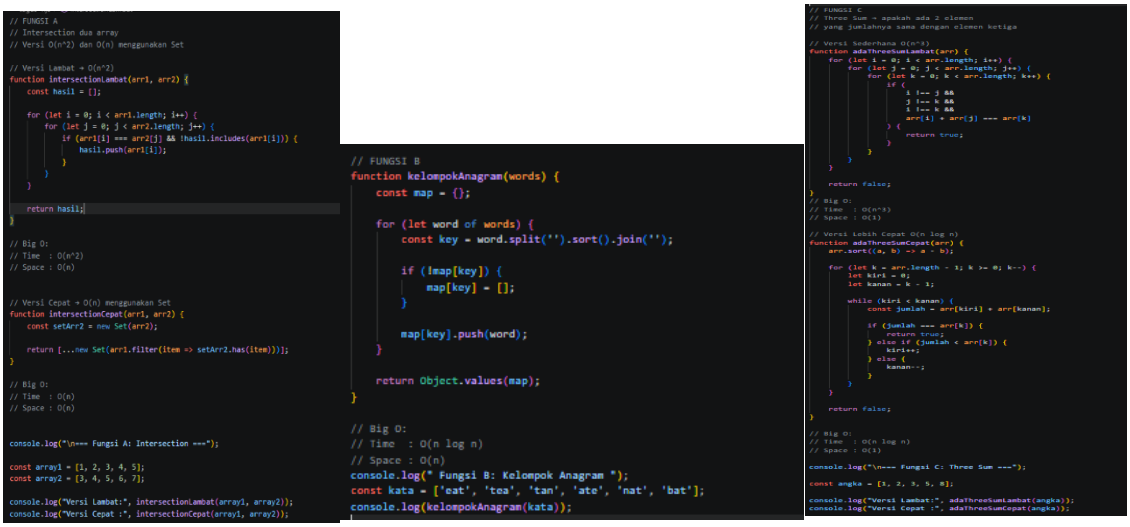
Gambar 3.2.5.1 Koding Refactor

```
duplikat lambat O(n^2) :  0 ms
duplikat cepat O(n) : 0 ms
frekuensi : { a: 3, b: 2, c: 1, d: 1 }
```

Gambar 3.2.5.2 Hasil Kod*ing*

3.3 Tugas

3.3.1 Analisis dan Refactor

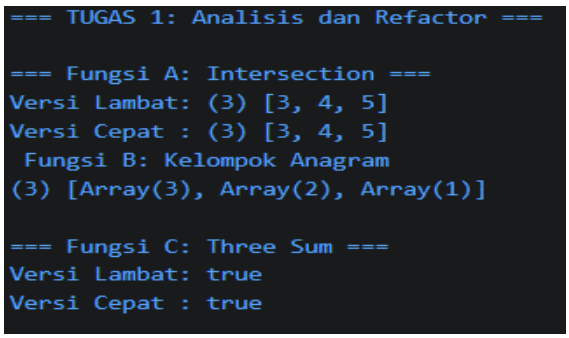


```jsx
// FUNGSI A
// Intersection dua array
// Versi O(n^2) dan O(n) menggunakan Set

// Versi Lambat → O(n^2)
function intersectionLambat(arr1, arr2) {
    const hasil = [];

    for (let i = 0; i < arr1.length; i++) {
        for (let j = 0; j < arr2.length; j++) {
            if (arr1[i] === arr2[j] && !hasil.includes(arr1[i])) {
                hasil.push(arr1[i]);
            }
        }
    }

    return hasil;
}

// Big O:
// Time  : O(n^2)
// Space : O(n)


// Versi Cepat → O(n) menggunakan Set
function intersectionCepet(arr1, arr2) {
    const setArr2 = new Set(arr2);

    return [...new Set(arr1.filter(item => setArr2.has(item)))];
}

// Big O:
// Time  : O(n)
// Space : O(n)


console.log("\n=== Fungsi A: Intersection ===");

const array1 = [1, 2, 3, 4, 5];
const array2 = [3, 4, 5, 6, 7];

console.log("Versi Lambat:", intersectionLambat(array1, array2));
console.log("Versi Cepat :", intersectionCepat(array1, array2));
```

```jsx
// FUNGSI B
function kelompokAnagram(words) {
    const map = {};

    for (let word of words) {
        const key = word.split('').sort().join('');

        if (!map[key]) {
            map[key] = [];
        }

        map[key].push(word);
    }

    return Object.values(map);
}

// Big O:
// Time  : O(n log n)
// Space : O(n)
console.log(" Fungsi B: Kelompok Anagram ");
const kata = ['eat', 'tea', 'tan', 'ate', 'nat', 'bat'];
console.log(kelompokAnagram(kata));
```

```jsx
// FUNGSI C
// Three Sum - apakah ada 2 elemen
// yang jumlahnya sama dengan elemen ketiga

// Versi Sederhana O(n^3)
function adaThreeSumLambat(arr) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = 0; j < arr.length; j++) {
            for (let k = 0; k < arr.length; k++) {
                if (
                    i !== j &&
                    j !== k &&
                    i !== k &&
                    arr[i] + arr[j] === arr[k]
                ) {
                    return true;
                }
            }
        }
    }

    return false;
}
// Big O:
// Time  : O(n^3)
// Space : O(1)

// Versi Lebih Cepat O(n log n)
function adaThreeSumCepat(arr) {
    arr.sort((a, b) => a - b);

    for (let k = arr.length - 1; k >= 0; k--) {
        let kiri = 0;
        let kanan = k - 1;

        while (kiri < kanan) {
            const jumlah = arr[kiri] + arr[kanan];

            if (jumlah === arr[k]) {
                return true;
            } else if (jumlah < arr[k]) {
                kiri++;
            } else {
                kanan--;
            }
        }
    }

    return false;
}

// Big O:
// Time  : O(n log n)
// Space : O(1)

console.log("\n=== Fungsi C: Three Sum ===");

const angka = [1, 2, 3, 5, 8];

console.log("Versi Lambat:", adaThreeSumLambat(angka));
console.log("Versi Cepat :", adaThreeSumCepat(angka));
```

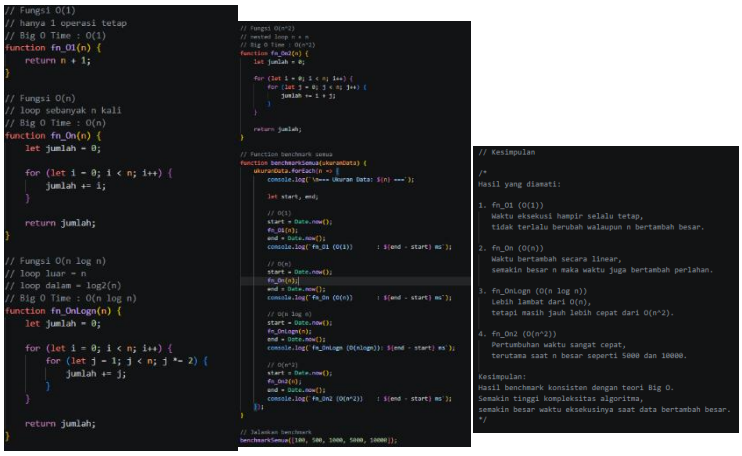
Gambar 3.3.1.1 Koding Analisis dan Refactor



```
=== TUGAS 1: Analisis dan Refactor ===

=== Fungsi A: Intersection ===
Versi Lambat: (3) [3, 4, 5]
Versi Cepat : (3) [3, 4, 5]
 Fungsi B: Kelompok Anagram
(3) [Array(3), Array(2), Array(1)]

=== Fungsi C: Three Sum ===
Versi Lambat: true
Versi Cepat : true
```

Gambar 3.3.1.2 Hasil Kod*ing*

3.3.2 Visualisasi Pertembuhan Big O

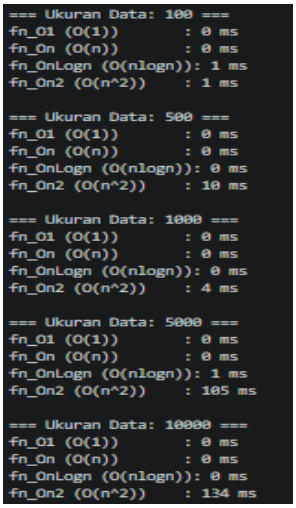


```
// Fungsi O(1)
// hanya 1 operasi tetap
// Big O Time : O(1)
function fn_O1(n) {
    return n + 1;
}

// Fungsi O(n)
// loop sebanyak n kali
// Big O Time : O(n)
function fn_On(n) {
    let jumlah = 0;

    for (let i = 0; i < n; i++) {
        jumlah += i;
    }

    return jumlah;
}

// Fungsi O(n log n)
// loop luar - n
// loop dalam - log2(n)
// Big O Time : O(n log n)
function fn_OnLogn(n) {
    let jumlah = 0;

    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) {
            jumlah += j;
        }
    }

    return jumlah;
}
```

```
// Fungsi O(n^2)
// nested loop n x n
// Big O Time : O(n^2)
function fn_On2(n) {
    let jumlah = 0;

    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            jumlah += i * j;
        }
    }

    return jumlah;
}

// Function benchmark semua
function benchmarkSemua(ukuranData) {
    ukuranData.forEach(n => {
        console.log(`\n=== Ukuran Data: ${n} ===`);

        let start, end;

        // O(1)
        start = Date.now();
        fn_O1(n);
        end = Date.now();
        console.log(`fn_O1 (O(1))      : ${end - start} ms`);

        // O(n)
        start = Date.now();
        fn_On(n);
        end = Date.now();
        console.log(`fn_On (O(n))      : ${end - start} ms`);

        // O(n log n)
        start = Date.now();
        fn_OnLogn(n);
        end = Date.now();
        console.log(`fn_OnLogn (O(nlogn)): ${end - start} ms`);

        // O(n^2)
        start = Date.now();
        fn_On2(n);
        end = Date.now();
        console.log(`fn_On2 (O(n^2))    : ${end - start} ms`);
    });
}

// Jalankan benchmark
benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

```
// Kesimpulan
/*
Hasil yang diamati:

1. fn_O1 (O(1))
   Waktu eksekusi hampir selalu tetap,
   tidak terlalu berubah walaupun n bertambah besar.

2. fn_On (O(n))
   Waktu bertambah secara linear,
   semakin besar n maka waktu juga bertambah perlahan.

3. fn_OnLogn (O(n log n))
   Lebih lambat dari O(n),
   tetapi masih jauh lebih cepat dari O(n^2).

4. fn_On2 (O(n^2))
   Pertumbuhan waktu sangat cepat,
   terutama saat n besar seperti 5000 dan 10000.

Kesimpulan:
Hasil benchmark konsisten dengan teori Big O.
Semakin tinggi kompleksitas algoritma,
semakin besar waktu eksekusinya saat data bertambah besar.
*/
```

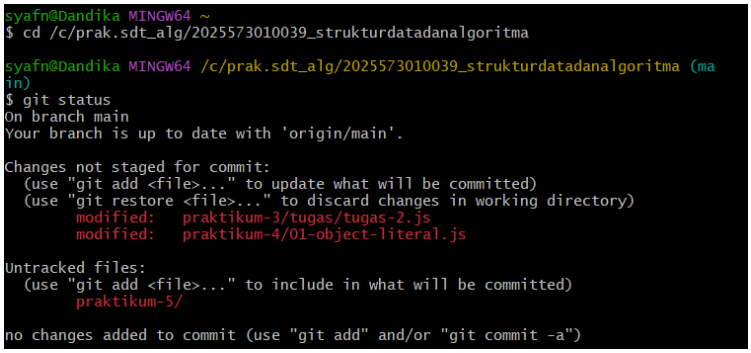
Gambar 3.3.2.1 Tugas Visualisasi Pertumbuhan Big O



```
=== Ukuran Data: 100 ===
fn_O1 (O(1))       : 0 ms
fn_On (O(n))       : 0 ms
fn_OnLogn (O(nlogn)): 1 ms
fn_On2 (O(n^2))    : 1 ms

=== Ukuran Data: 500 ===
fn_O1 (O(1))       : 0 ms
fn_On (O(n))       : 0 ms
fn_OnLogn (O(nlogn)): 0 ms
fn_On2 (O(n^2))    : 10 ms

=== Ukuran Data: 1000 ===
fn_O1 (O(1))       : 0 ms
fn_On (O(n))       : 0 ms
fn_OnLogn (O(nlogn)): 0 ms
fn_On2 (O(n^2))    : 4 ms

=== Ukuran Data: 5000 ===
fn_O1 (O(1))       : 0 ms
fn_On (O(n))       : 0 ms
fn_OnLogn (O(nlogn)): 1 ms
fn_On2 (O(n^2))    : 105 ms

=== Ukuran Data: 10000 ===
fn_O1 (O(1))       : 0 ms
fn_On (O(n))       : 0 ms
fn_OnLogn (O(nlogn)): 0 ms
fn_On2 (O(n^2))    : 134 ms
```

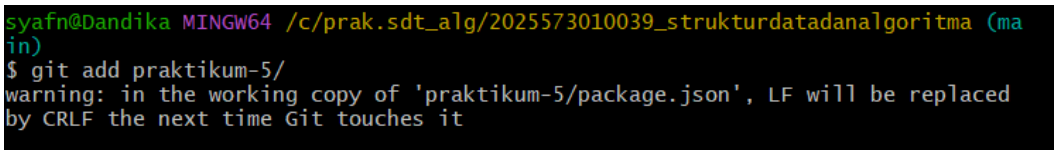
Gambar 3.3.2.2 Hasil Kod*ing*

3.4 Mengirim Praktikum Ke *GitHub*



```
syafn@Dandika MINGW64 ~
$ cd /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma

syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (ma
in)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   praktikum-3/tugas/tugas-2.js
        modified:   praktikum-4/01-object-literal.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        praktikum-5/

no changes added to commit (use "git add" and/or "git commit -a")
```

Gambar 3.4.1 Melihat Status *Git*



```
syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (ma
in)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced
by CRLF the next time Git touches it
```

Gambar 3.4.2 Menambahkan Praktikum-5 ke dalam *Git*

```
syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (ma
in)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main cd96526] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 586 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js
```

Gambar 3.4.3 Membuat *Folder* Untuk Mengumpulkan *File-File* Yang di Perlukan

```
syafn@Dandika MINGW64 /c/prak.sdt_alg/2025573010039_strukturdatadanalgoritma (ma
in)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.87 KiB | 500.00 KiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/adindariesyaputri/2025573010039_strukturdatadanalgoritma.g
it
   ef0e937..cd96526  main -> main
```

Gambar 3.4.4 *Push Folder* ke *GitHub*

praktikum-5	Praktikum 5: Analisis Kompleksitas Algoritma	4 minutes ago

Gambar 3.4.5 *Folder* Praktikum-5 Berhasil di-Unggah ke *GitHub*

BAB IV

PEMBAHASAN DAN ANALISA

4.1. Alat dan Bahan

Pada praktikum ini, alat dan bahan yang digunakan meliputi Visual Studio Code, Git Bash, dan modul praktikum. Visual Studio Code digunakan sebagai text editor untuk menulis dan menjalankan program JavaScript. Aplikasi ini dipilih karena ringan, mudah digunakan, serta memiliki banyak fitur pendukung seperti debugging dan extension.

Git Bash digunakan sebagai terminal untuk menjalankan perintah dalam pembuatan folder, file, serta pengelolaan repository. Selain itu, modul praktikum digunakan sebagai panduan utama dalam memahami konsep analisis kompleksitas algoritma serta langkah-langkah pengerjaan praktikum..

4.2. Pembuatan File dan Folder

Pada tahap ini dilakukan pembuatan struktur folder dan file menggunakan Git Bash. Langkah awal adalah membuka Git Bash dan masuk ke dalam repository menggunakan perintah cd. Setelah itu dibuat folder praktikum menggunakan perintah mkdir praktikum-5, kemudian masuk ke dalam folder tersebut.

Selanjutnya dilakukan inisialisasi project menggunakan npm init -y untuk membuat file package.json. Setelah itu dibuat file JavaScript menggunakan perintah touch. Tahapan ini bertujuan untuk mempersiapkan lingkungan kerja serta memastikan file tersusun dengan rapi dan terstruktur

4.3. Percobaan dan Latihan

4.3.1 Intro Kompleksitas

Pada percobaan ini diperkenalkan konsep dasar analisis kompleksitas algoritma. Program sederhana digunakan untuk melihat jumlah operasi yang dilakukan terhadap input tertentu. Dari percobaan ini dapat dipahami bahwa semakin besar input, maka jumlah operasi juga meningkat sesuai dengan pola tertentu.

4.3.2 Kelas *Big O*

Pada bagian ini dilakukan pengujian beberapa contoh algoritma dengan kompleksitas berbeda seperti O(1), O(n), dan $O(n^2)$. Hasil percobaan menunjukkan bahwa algoritma dengan kompleksitas kecil memiliki performa yang lebih baik dibandingkan algoritma dengan kompleksitas besar, terutama saat ukuran data meningkat.

4.3.3 Latihan 1

Latihan ini bertujuan untuk mengidentifikasi kompleksitas dari sebuah program. Dengan menganalisis jumlah perulangan dan operasi yang dilakukan, dapat ditentukan apakah program tersebut termasuk dalam kompleksitas linear, kuadratik, atau lainnya.

4.3.4 *Space Komplexity*

Pada percobaan ini dianalisis penggunaan memori dalam suatu program. Program yang menggunakan variabel sederhana memiliki kompleksitas O(1), sedangkan program yang membuat struktur data tambahan seperti array memiliki kompleksitas O(n). Hal ini menunjukkan bahwa penggunaan memori juga penting untuk diperhatikan selain waktu eksekusi.

4.3.5 *Refactcoring*

Tahap ini dilakukan perbaikan kode (refactoring) untuk meningkatkan efisiensi algoritma. Program yang awalnya memiliki kompleksitas tinggi diperbaiki menjadi lebih optimal tanpa mengubah hasil akhir. Refactoring membantu mengurangi operasi yang tidak perlu sehingga program menjadi lebih cepat dan efisien.

4.4. Tugas

4.4.1 Analisis dan Refactoring

Pada tugas ini dilakukan analisis terhadap sebuah program untuk menentukan kompleksitasnya. Setelah itu dilakukan refactoring untuk mengoptimalkan kode. Hasil menunjukkan bahwa setelah dilakukan perbaikan, kompleksitas program menjadi lebih rendah sehingga performa meningkat.

4.4.2 Visualisasi Pertumbuhan Big O

Pada tugas ini dibuat visualisasi pertumbuhan kompleksitas algoritma. Visualisasi ini menunjukkan bagaimana waktu eksekusi meningkat seiring dengan bertambahnya input. Dari hasil tersebut terlihat bahwa algoritma dengan kompleksitas eksponensial meningkat jauh lebih cepat dibandingkan algoritma linear atau logaritmik.

4.5. Mengirim Praktikum ke Github

Tahap terakhir adalah mengunggah hasil praktikum ke GitHub. Langkah-langkah yang dilakukan yaitu:

1. Mengecek status file dengan git status.
2. Menambahkan file dengan git add.
3. Melakukan commit dengan git commit -m.
4. Mengunggah ke repository dengan git push.

Setelah proses selesai, file praktikum-5 berhasil diunggah dan tersimpan pada repositori GitHub.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma sangat penting dalam menentukan efisiensi suatu program. Dengan menggunakan Big O Notation, programmer dapat mengetahui performa algoritma berdasarkan pertumbuhan input tanpa bergantung pada perangkat keras.

Time complexity digunakan untuk mengukur kecepatan eksekusi program, sedangkan space complexity digunakan untuk mengukur penggunaan memori. Keduanya harus diperhatikan agar program tidak hanya cepat tetapi juga hemat sumber daya.

Selain itu, refactoring kode terbukti mampu meningkatkan efisiensi program dengan mengurangi kompleksitas yang tidak perlu. Dengan memilih algoritma yang tepat dan melakukan optimasi, program dapat berjalan lebih optimal terutama pada data berukuran besar.

5.2 Saran

Adapun saran sebagai berikut:

1. Memahami konsep dasar analisis kompleksitas sebelum mengimplementasikan program.
2. Melakukan latihan secara rutin agar lebih terbiasa dalam menentukan kompleksitas algoritma.
3. Melakukan refactoring kode secara berkala untuk meningkatkan performa program.
4. Menggunakan algoritma yang efisien sesuai dengan kebutuhan program.

DAFTAR PUSTAKA

GeeksforGeeks. (2024). *Time Complexity and Space Complexity*. Diakses dari: https://www.geeksforgeeks.org/dsa/time-complexity-and-space-complexity/

Jakarta Telkom University. (2024). *Analisis Kompleksitas Algoritma: Memahami Big O Notation*. Diakses dari: https://jakarta.telkomuniversity.ac.id/analisis-kompleksitas-algoritma-memahami-big-o-notation/

Koding Data. (2024). *Memahami Big O Notation: Panduan Lengkap*. Diakses dari: https://www.kodingdata.com/news/memahami-big-o-notation-panduan-lengkap/

Phisoft. (2024). *Teknik Code Refactoring untuk Peningkatan Performa Aplikasi*. Diakses dari: https://phisoft.co.id/blog/teknik-code-refactoring-untuk-peningkatan-performa-aplikasi/

Repository Universitas Pendidikan Indonesia. (2012). *Analisis Kompleksitas Algoritma*. Diakses dari: https://repository.upi.edu/2878/6/S_MTK_0900249_CHAPTER3.pdf

Zulman, M. R. (2026). *Modul Praktikum Struktur Data dan Algoritma Modul 5: Analisis Kompleksitas Algoritma*. Program Studi Teknik Informatika.